

Two structures to build

A dynamic array, and a linked list

Two exercises, with staged hints. One skeleton to fill in, one broken program to repair.

Both are built out of nothing but structs, pointers and malloc — everything you already have.

THE TWO FILES

```
dynamic_array.c    # fill in  
linked_list.c      # repair  
  
$ gcc dynamic_array.c -Wall \  
    -Wextra -g -o dynarr  
$ ./dynarr
```

Why these two

Neither exercise introduces anything new. They ask you to combine things you met one at a time in files 01 to 07, and that combination is where C actually gets difficult.

Structures

Grouping a pointer and two counters into one thing that travels together.

Pointers

Functions that modify the caller's data, and pointers that live inside structs.

malloc, realloc, free

Deciding when memory is yours, when it grows, and when it stops being yours.

No safety net

No bounds check, no garbage collector, no exception. Only what you write.

Exercise A · dynamic array

An array that grows as you add to it. This is roughly what a Python list is underneath. You are given a skeleton with three empty functions.

Exercise B · linked list

A chain of separately allocated nodes. You are given a program that compiles, appears to run, and is wrong in at least four ways.

How to work on these

1

Run it first

Even a broken program tells you something. Compile and run before you change a single character, and keep that output to compare against.

2

Draw it on paper

Both structures are pictures before they are code. Three boxes and two arrows will answer most of your questions faster than the debugger.

3

Build with the sanitizer

`gcc -Wall -Wextra -g -fsanitize=address`. For these two exercises it is not optional: most of the mistakes you will make are invisible without it.

4

One change, one compile

The same loop as before. Change one line, predict the result, recompile, compare. Two changes at once and you learn nothing from the crash.

● ABOUT THE HINTS IN THIS DECK

- Each exercise has its hints numbered and ordered. They get more explicit as they go, and the last ones contain code with blanks in it.
- Try the exercise first, then take the hints one at a time and stop at the first one that unblocks you. Nothing here is a finished solution — the blanks are yours to fill.



Dynamic array

An array of int that grows on demand. You are given the struct, three empty functions and a main() that already says exactly how they must behave.

- Read the struct and work out what each field is for
- Implement `initialize_arr`, `insert_arr` and `free_arr`
- Decide how much bigger to make the block each time
- Fix the two mistakes that are already in `main()`

The problem it solves

What you cannot do

```
int arr[7];
```

Seven, decided when you wrote the line, and seven forever. If the data arrives from a file or from the user, you do not know the number at compile time — and there is no way to make this array bigger later.

What realloc alone does not solve

realloc can grow a malloc'd block. But calling it once per inserted element means copying the whole array again and again, and you would still have to remember the current length in a separate variable that nothing keeps in sync.

● THE ANSWER, IN ONE SENTENCE

Keep the block, its capacity and the number of elements actually used together inside one struct, and grow the block in large jumps rather than one element at a time.

That struct is what a Python list, a C++ `std::vector` and a Java `ArrayList` all are underneath. You are about to write one in roughly thirty lines.

The skeleton you are given

THE STRUCT AND THE THREE STUBS

```
#include <stdio.h>
#include <stdlib.h>

typedef struct dynArray DynArray;
struct dynArray{
    int* array;    // a dynamic array of int
    size_t used;   // the number of used entries
    size_t size;   // the memory allocated to it
};

void initialize_arr(DynArray* da, size_t size){
    /* Initialize the dynamic array with "size" elements */
}

void insert_arr(DynArray* da, int element) {
    // check if array is full, then "extend" the array
    // update the size record
    // apply realloc with the updated size value
    // add element at the next index
}
```

```
void free_arr(DynArray* da) {
}
```

AND THE main() IT MUST SATISFY

```
int main(){
    DynArray* dynarr;
    initialize_arr(dynarr, 7);
    for (int i = 0; i < 1000; i++)
        insert_arr(dynarr, i);

    printf("%d\n", dynarr->array[12]);
    printf("%d\n", dynarr->used);
    free_arr(dynarr);

    return 0;
}
```

● READ main() BEFORE YOU WRITE ANYTHING

It is a specification: seven elements to start, a thousand inserted, element 12 readable afterwards. It also contains two mistakes of its own.

Three fields, and one rule that must never break

int* array

The address malloc gave you. Every element lives inside that one block, side by side.

size_t used

How many elements you have actually stored. This is the number anyone using your array cares about.

size_t size

How many would fit before you must call realloc. The capacity — normally larger than used.

BEFORE CODING

Write down, for each of the three functions, what used and size must be when it returns. That is your specification, and it is shorter than the code.

array



used = 3

spare capacity

size = 8

THE INVARIANT

$0 \leq \text{used} \leq \text{size}$, at every moment between your function calls. Every function you write must leave that true. The day you find used larger than size, you have already written past the end of the block.

Your task: three functions

Do not change the signatures. Each one is described below by what must be true after it returns, not by how to write it.

initialize_arr

```
void initialize_arr(DynArray* da, size_t  
size)
```

AFTERWARDS

da->array points at a block with room for size
ints.

da->used is 0 — the array is empty, not full.

da->size records that capacity.

insert_arr

```
void insert_arr(DynArray* da, int  
element)
```

AFTERWARDS

element sits at the first free index.

used has grown by exactly one.

The invariant still holds: if the block was full, it
was enlarged first.

free_arr

```
void free_arr(DynArray* da)
```

AFTERWARDS

The block is back with the system.

Nothing left in da can be dereferenced by
accident.

Calling it does not free the struct itself.

TRY IT

Start with initialize_arr and free_arr, and give insert_arr a body that only works while there is spare room. Compile and run with a loop of 5 inserts instead of 1000. Only then write the growing part.

Before you start: main() is already broken

NOTHING WAS EVER ALLOCATED FOR THE STRUCT ITSELF

```
DynArray* dynarr;           // an uninitialised pointer
initialize_arr(dynarr, 7);  // ...passed straight in
```

dynarr holds whatever was on the stack. The first `da->array = ...` inside your function writes through it, to an address nobody chose. Exactly the mistake from file 01, one level deeper.

FIX 1: PUT THE STRUCT ON THE STACK

```
DynArray dynarr;
initialize_arr(&dynarr, 7);
insert_arr(&dynarr, i);
printf("%d\n", dynarr.array[12]); // dot, not arrow
```

FIX 2: PUT IT ON THE HEAP, AND KEEP main() AS IT IS

```
DynArray* dynarr = malloc(sizeof(DynArray));
/* ... */
free_arr(dynarr); // frees the block of ints
free(dynarr);     // frees the struct itself
```

THE SECOND MISTAKE

`printf("%d\n", dynarr->used)` prints a `size_t` with the specifier for `int`. On a 64-bit machine those are different sizes. The right one is `%zu`.

WHICH FIX SHOULD YOU PICK?

- Either is correct. Fix 1 is simpler and needs no `free` for the struct.
- Fix 2 keeps `main()` exactly as written and is what a real library does, because the struct then outlives the function that created it.
- Whichever you choose, be able to say who allocates and who frees.

Hint 1 — initialize_arr

FILL IN THE BLANKS

```
void initialize_arr(DynArray* da, size_t size){
    da->array = malloc(_____ * sizeof(_____));
    if (da->array == NULL) {
        /* and now what? */
    }
    da->used = _____;
    da->size = _____;
}
```

● TRY IT

Write only this function, then in main print da->used and da->size straight after calling it. Two numbers, 0 and 7. If you cannot get those two right, nothing later will work.

malloc takes bytes

Not a number of elements. Multiply by sizeof(int) — or better, by sizeof(*da->array), which stays correct if you ever change the element type.

used starts at zero

The array has room for size elements. It contains none of them. Confusing capacity with content is the single most common error here.

What if malloc fails

It returns NULL. This function returns void, so it cannot report the failure. Decide now: print to stderr and exit, or change the signature to return int.

Hint 2 — insert_arr has two halves

FILL IN THE BLANKS

```
void insert_arr(DynArray* da, int element) {  
  
    if (da->used == da->size) {    // half one: grow  
        da->size = ____;  
        da->array = realloc(da->array, ____);  
    }  
  
    da->array[____] = element;    // half two: store  
    da->used ____;
```

● WATCH OUT

da->array = realloc(da->array, n) is the convenient form, and it leaks the whole array if realloc returns NULL — you have overwritten your only pointer to it. Assign to a temporary, test it, then commit.

Grow first, store second

Never the other way round. The test is == because used can only ever reach size, one insert at a time.

realloc wants bytes too

The second argument is the new size in bytes, so it is the new capacity times sizeof(int) — not the capacity on its own.

Assign the result back

realloc may move the block somewhere else entirely and free the old address. Ignore the return value and your pointer is dangling.

The last two lines

They can be written as one. Once you see how, you will also see why used must be incremented after it is used as the index.

Hint 3 — how much bigger?

The blank on the `da->size` line is the only real design decision in this exercise. Starting from a capacity of 7 and inserting the thousand elements `main()` asks for:

993

reallocs

if you grow by one element each time the array is full. Every one of them may copy the entire array to a new address.

8

reallocs

if you double the capacity instead: 7, 14, 28, 56, 112, 224, 448, 896. The copying cost, spread over the inserts, becomes negligible.

● THE IDEA BEHIND IT

Each doubling costs a copy proportional to the array, but it also buys you as many free inserts as the array is long. Averaged out, one insert costs a constant amount of work however big the array gets.

● TWO EDGE CASES

- Doubling a capacity of 0 gives 0. If you allow an empty array, the growth rule needs a floor.
- Growth factors other than 2 are perfectly reasonable. 1.5 wastes less memory. Try both and measure.

Hint 4 — free_arr, and what free does not do

FILL IN THE BLANKS

```
void free_arr(DynArray* da) {  
    free(____);  
    da->array = ____;  
    da->used  = ____;  
    da->size  = ____;  
}
```

● free() DOES NOT TOUCH YOUR POINTER

It hands the block back to the allocator and returns. `da->array` still holds the old address afterwards, and it still looks perfectly valid. Setting it to `NULL` is what turns a silent use-after-free into an immediate, obvious crash.

● TRY IT

Call `free_arr` twice in a row on the same array. With the `NULL` assignment in place, what happens? Without it? `free(NULL)` is explicitly allowed and does nothing, which is why the assignment makes the second call harmless.

The three assignments after the `free` are not decoration. They restore the struct to the state it had before `initialize_arr`, which means a later `insert_arr` on the same struct fails loudly instead of writing into memory that now belongs to something else.

● WHO FREES THE STRUCT?

`free_arr` releases the block of ints. It cannot release the `DynArray` itself: it does not know whether that came from `malloc` or from the stack. Whoever allocated it frees it.

Does it work?

WHAT main() SHOULD PRINT

```
$ gcc dynamic_array.c -Wall -Wextra -g -o dynarr
$ ./dynarr
12
1000
```

12 because element *i* was inserted with the value *i*, and 1000 because that is how many went in. If the second line is a huge number or a negative one, you are still printing a `size_t` with `%d`.

THEN THE ONE THAT MATTERS

```
$ gcc dynamic_array.c -Wall -Wextra -g \
    -fsanitize=address -o dynarr
$ ./dynarr           # silence here means silence
```

Four tests worth adding

- Initialise with a capacity of 1, then insert 1000. Same two numbers?
- Initialise, then free immediately without inserting anything.
- Print used and size after every insert for the first 20. Watch the capacity jump.
- Insert 1000, free, then insert again. It should fail obviously, not quietly.

● IF YOU HAVE `valgrind`

`valgrind ./dynarr` should report all heap blocks freed. One block still reachable usually means you forgot the struct in Fix 2.

Going further with the dynamic array

1

Safe access

Add `int get_arr(DynArray* da, size_t index)` that refuses an index $\geq$ used instead of reading past the end. You are adding the bounds check C never gave you.

2

Remove and shrink

Add `remove_arr` taking an index, shifting the tail down. Then halve the capacity when used drops below a quarter of it — and work out why a quarter, not a half.

3

A constructor

Replace `initialize_arr` with `DynArray* new_arr(size_t size)` that allocates the struct too and returns it. `main()` then never touches `malloc` directly.

4

Any type at all

Store the element size in the struct and make array a `void*`. Suddenly you need `memcpy` from file 03, and indexing becomes arithmetic on bytes.

5

Report failure

Change the functions to return `int`, 0 for success. Then make `main()` actually check them. Notice how much more code correct error handling is.

6

Measure it

Time 10 million inserts with doubling, with 1.5x, and with +1 growth. The difference is the point of hint 3, in seconds.

Checklist: exercise A

You are finished when you can answer yes to all of these without checking the code again.



main() runs and prints 12 then 1000

with -Wall -Wextra and not one warning.



free_arr leaves nothing dereferenceable

calling it twice does not crash.



The sanitizer build is silent

no overflow, no leak, no use after free.



You can name who frees the struct

and it is not free_arr.



used and size are never confused

you can say which one main() prints, and why.



The capacity grows in jumps

and you can justify the factor you picked.



realloc's return value is assigned back

and you know what happens if it returns NULL.



It still works from a capacity of 1

your growth rule has no hole at the bottom.



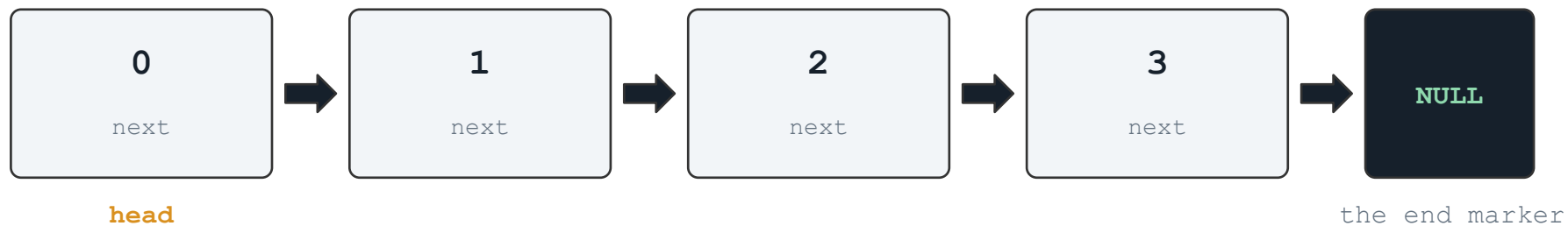
Linked list

A chain of separately allocated nodes, each one holding a value and the address of the next. You are given a program that compiles, appears to run, and is wrong in at least four ways.

- Run it, then draw on paper what it actually builds
- Decide what marks the end of the list, and apply it everywhere
- Make `print_list` show every value exactly once
- Make `erase_list` free every node, and only once

What a linked list is

No single block, no capacity, no realloc. Each node is its own small allocation, and every node knows only where the next one lives.



Growing costs nothing

Adding a node is one malloc and one pointer assignment. Nothing is ever copied, and there is no capacity to run out of.

Reaching element 500

means walking 500 nodes. There is no arithmetic that jumps straight to it, because the nodes are scattered across the heap.

Every node is a decision

One malloc per node, one free per node. Lose the address of a node and you have lost everything after it too.

NOTE

The struct is recursive: a node contains a pointer to a node. That is only possible because a pointer has a fixed size whatever it points at — the compiler does not need to know how big a Node is to reserve room for its address.

Against the dynamic array you just wrote

	Dynamic array	Linked list
Read element i	one addition, instantly	walk i nodes from the head
Insert at the front	shift every element along	one malloc, two assignments
Insert at the end	free, until the block fills up	one malloc, if you kept a tail pointer
Memory layout	one contiguous block, cache friendly	scattered nodes, plus 8 bytes of pointer each
Growing	realloc, which may copy everything	nothing to copy, ever
Freeing	one call to free	one call per node, in the right order

Neither wins. The array is faster at almost everything until you have to insert in the middle of a long list, and the list never has to copy. Knowing which operation costs what is the whole reason for writing both by hand.

The code you are given (1 of 2)

THE NODE, AND THE main() THAT BUILDS THE CHAIN

```
typedef struct node Node;
struct node{
    int data;
    Node* next;
};

int main(){
    Node* first_node = (Node*) malloc(1 * sizeof(Node));
    Node* current_node = first_node;

    for (int value = 0; value < 9; value++){
        current_node->data = value;
        current_node->next = (Node*) malloc(1 * sizeof(Node));
        current_node = current_node->next;
    }

    print_list(first_node);
    erase_list(first_node);
    print_list(first_node);
}
```

Read the loop body in order, then count. How many times does the loop run, how many nodes does it allocate, and how many of them are ever given a value?

The two numbers are not the same. That gap is where the first bug lives.

DO THIS FIRST

- Compile and run it. Then run it again. And a third time.
- Now compile with -fsanitize=address and run it once more.
- Write down what you saw each time, before reading any hint.

The code you are given (2 of 2)

print_list

```
void print_list(Node* head){
    Node* current_node = head;
    while (current_node->next != NULL){
        printf("%d, ", current_node->data);
        current_node = current_node->next;
    }
    printf("\n");
}
```

erase_list

```
void erase_list(Node* head){
    Node* current_node = head;
    Node* previous_node;
    while (current_node->next != NULL){
        previous_node = current_node;
        current_node = current_node->next;
        free(previous_node);
    }
}
```

● TWO QUESTIONS BEFORE YOU TOUCH ANYTHING

- Both loops test the same condition. Take a list of exactly three nodes, ending in NULL, and write down on paper which nodes each loop visits. Which one is left out, and why?
- erase_list reads current_node->next and then frees a node. Check the order carefully: is it reading through memory it has already given back, or not? Being able to answer that precisely is most of this exercise.

Note that neither function is told how long the list is. The only thing that can stop either loop is a pointer that equals NULL, so something, somewhere, has to put one there.

Your task

The program compiles without a single warning, even with `-Wall -Wextra`. On many machines it appears to print the right thing. It is still wrong in at least four ways.

4

Free every node

`erase_list` must free each node exactly once, never read a node after freeing it, and leave nothing behind.

5

The last line

Make the final `print_list` call safe, or delete it and be able to explain why it could never have been safe.

6

Tidy up

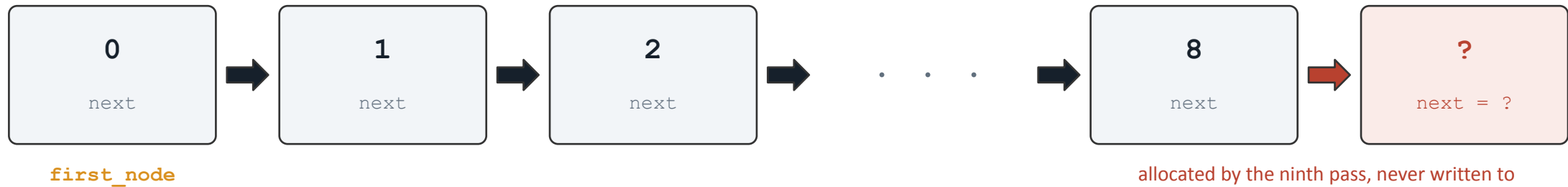
Deal with the smaller things in hint 5 once the four real bugs are gone.

● AT LEAST FOUR

The count is deliberately vague. Two of the bugs partly hide each other, so fixing one can make the output look worse before it looks better. Trust the sanitizer, not the appearance of the output.

Draw what main() actually builds

The loop runs nine times. It allocates ten nodes. The tenth never receives a value, and nothing ever sets its next pointer.



Where does the list end?

It does not. Both loops stop when they find a next pointer equal to NULL, and no line in the program ever assigns NULL to anything. What they are actually testing is whatever bytes malloc happened to hand back for that tenth node.

● WHY IT OFTEN LOOKS FINE ANYWAY

Fresh pages from the operating system usually arrive full of zeros, so that uninitialised next is very often NULL by accident, and the first print looks perfect. That is luck. It is also exactly the kind of bug that survives until somebody else runs your program.

Hint 2 — pick one convention and apply it three times

There are two reasonable answers. The file currently uses the loops of one of them and the construction of neither.

(a) The last node holds a value

Nine values means nine nodes. The last one's next is set to NULL explicitly.

Every loop then reads while (current != NULL).

This is what you will see everywhere else, and what the rest of this deck assumes.

(b) A trailing empty sentinel

Ten nodes, the last one carrying no value, with its next set to NULL explicitly.

Loops then read while (current->next != NULL), which is what the file already does.

Workable, but you now have a node that must never be printed and must still be freed.

● WHICHEVER YOU PICK

- Write the sentence down above your code: "the list ends at the node whose next is NULL" or "the list ends at the empty node whose next is NULL".
- Then check all three places against it: the construction loop in main, the loop in print_list, and the loop in erase_list. A bug in a linked list is almost always two of those three disagreeing.

Hint 3 — the loop that stops one node early

Take a list of exactly three nodes ending properly in NULL, and trace both conditions by hand. This is a thirty-second exercise on paper and it settles the question for good.

WHAT EACH CONDITION VISITS

```
list:  [A] -> [B] -> [C] -> NULL

while (current != NULL)
    visits A, B, C      <- all of them

while (current->next != NULL)
    visits A, B        <- C is skipped
```

NOTE

"Am I standing on a node?" and "is there a node after this one?" are different questions. Only the first one is a correct test for whether there is work left to do.

THE TWO BUGS CANCEL OUT

- With nine values, the file allocates a tenth node. `print_list` then stops one node early — at the tenth. So it prints exactly nine values and looks correct.
- Fix only the extra node and the output loses a value. Fix only the loop condition and it prints garbage from the tenth. You have to fix both, together, which is why hint 2 comes first.

TRY IT

Once fixed, test `print_list` on a list of one node, and on a head that is NULL. Both must work without a special case in main.

Hint 4 — erase_list frees what it still needs

As written, the loop is careful: it moves on before freeing. But it inherits the same condition as print_list, so it stops one node early — and that node is leaked on every single run.

FILL IN THE BLANKS

```
void erase_list(Node* head) {  
    Node* current = head;  
    while (current != NULL) {  
        Node* next = ____; /* read BEFORE free */  
        free(____);  
        current = ____;  
    }  
}
```

● THE ORDER IS THE WHOLE POINT

free(current) then current = current->next reads a field of a block that no longer belongs to you. The value is often still there, so it usually appears to work. The sanitizer calls it heap-use-after-free and it is one of the nastiest bugs in C.

How you will know

```
==1==ERROR: AddressSanitizer:  
    heap-use-after-free on address ...  
READ of size 8 at 0x... thread T0  
    #0 in erase_list linked_list.c:24  
freed by thread T0 here:  
    #0 in free
```

● AND FOR THE LEAK

A clean run under -fsanitize=address ends with nothing at all. One node left behind is reported as a direct leak of 16 bytes, with the malloc that created it.

Hint 5 — the last two lines of main()

```
erase_list(first_node);  
print_list(first_node); // every node is gone
```

`first_node` still holds the address it always held. The memory at that address is no longer yours. `erase_list` cannot have fixed this: it was handed a copy of the pointer, so anything it assigns to its own parameter dies with the function.

IF YOU WANT `erase_list` TO CLEAR THE CALLER'S POINTER

```
void erase_list(Node** head) {  
    /* ... free every node ... */  
    *head = NULL;  
}  
  
erase_list(&first_node); // at the call site
```

● THE SIMPLE FIX FIRST

- Set `first_node = NULL`; yourself, straight after the call.
- Then make `print_list` survive a NULL head by printing nothing — which it will already do, once its loop tests `current != NULL`.
- The second print then produces an empty line, which is the correct answer for an empty list.

● SMALLER THINGS, ONCE THE REST WORKS

- The `(Node*)` cast on `malloc` is unnecessary in C.
- `malloc` can return `NULL`. Nothing here checks it.
- `print_list` leaves a trailing comma. Decide what you want.
- `main` has no `return 0`.

Going further with the linked list

1

Build it properly

Write `Node* push_front(Node* head, int value)` returning the new head, and rebuild the nine-element list with a loop that calls it. Notice the list comes out backwards.

2

Ask it questions

Add `size_t length(Node* head)` and `Node* find(Node* head, int value)`. Both are the same walk you have already written twice.

3

Delete one node

`Node* remove_value(Node* head, int value)`. The interesting case is when the node to remove is the head itself, which is why the function has to return the new head.

4

Reverse it

In one pass, with three pointers and no new allocation. Harder than it sounds, and a classic interview question for exactly that reason.

5

Both directions

Add `Node* prev` to the struct and keep it consistent through every insert and delete. Twice the pointers, four times the mistakes.

6

Carry real data

Replace `int` data with the `Enemy` struct from file 01. Nothing about the list changes, which is the point of separating structure from payload.

Checklist: exercise B

You are finished when you can answer yes to all of these without checking the code again.



print_list shows all nine values

once each, with no garbage at either end.



print_list(NULL) prints an empty line

and does not crash or need a special case.



The sanitizer build is completely silent

no leak, no use after free, on every run.



No node is freed twice

and none is left behind.



You can state the end convention

in one sentence, and all three loops obey it.



The program is the same on every run

run it ten times and the output never changes.



Nothing reads a node after freeing it

you can point at the line that guarantees it.



You know why it looked right before

and can explain it to someone else.

How to check your work

Both exercises are memory exercises. Your eyes are not a memory checker; these tools are.

THE COMMANDS

```
$ gcc f.c -Wall -Wextra -g -O0 -o prog
$ ./prog

$ gcc f.c -Wall -Wextra -g \
    -fsanitize=address -o prog
$ ./prog          # silence = clean

$ valgrind --leak-check-full ./prog
```

● COMPILE WITH -g AND -O0

Without -g the sanitizer reports addresses instead of line numbers. Without -O0 the optimiser may move code around and make the trace harder to read.

What the messages mean

heap-buffer-overflow

You wrote outside the block. In exercise A: an index past used, or a realloc that did not actually happen.

heap-use-after-free

You touched memory you had already freed. In exercise B: reading next after free.

attempting double-free

The same block freed twice. Usually a missing NULL assignment after the first free.

detected memory leaks

Something you allocated was never freed. Read the stack trace: it names the malloc, not the leak.

One lesson, twice

Neither exercise is hard because of the algorithm. Both are hard because of one question you have to be able to answer about every line: who owns this block of memory right now, and when does it stop being theirs?

- Every malloc has exactly one matching free, and you can point at it
- Every pointer is either valid or NULL — never merely old
- Every loop over a structure agrees with how that structure ends
- Anything that only works some of the time does not work

WHEN YOU THINK YOU ARE DONE

```
$ gcc f.c -Wall -Wextra -g \
    -fsanitize=address -o prog
$ ./prog
$ ./prog
$ ./prog

# same output three times,
# and nothing from the
# sanitizer. Then you are done.
```